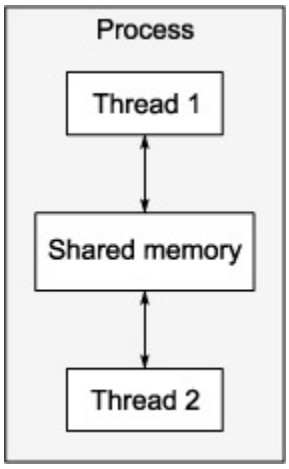
Using separate processes for concurrency also has an additional advantage—you can run the separate processes on distinct machines connected over a network. Though this increases the communication cost, on a carefully designed system it can be a cost-effective way of increasing the available parallelism and improving performance.

Concurrency with Multiple Threads

The alternative approach to concurrency is to run multiple threads in a single process. Threads are much like lightweight processes: each thread runs independently of the others, and each thread may run a different sequence of instructions. But all threads in a process share the same address space, and most of the data can be accessed directly from all threads—global variables remain global, and pointers or references to objects or data can be passed around among threads. Although it's often possible to share memory among processes, this is complicated to set up and often hard to manage, because memory addresses of the same data aren't necessarily the same in different processes. Figure 1.4 shows two threads within a process communicating through shared memory.

Figure 1.4. Communication between a pair of threads running concurrently in a single process



The shared address space and lack of protection of data between threads makes the overhead associated with using multiple threads much smaller than that from using multiple processes, because the operating system has less bookkeeping to do. But the flexibility of shared memory also comes with a price: if data is accessed by multiple threads, the application